

```

import aiohttp
import asyncio
import time
import ssl
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
URL = "https://speed.hetzner.de/10MB.bin"

async def download_file(session, i):
    print(f"Downloading file {i}...")
    async with session.get(URL, ssl=False) as response: # Disable SSL verification
        content = await response.read()
        print(f"Downloaded file {i}: {len(content)} bytes")

async def main():
    async with aiohttp.ClientSession() as session:
        tasks = [download_file(session, i) for i in range(10)]
        await asyncio.gather(*tasks)

start = time.time()
asyncio.run(main())
end = time.time()

print(f"Asynchronous downloading took {end - start:.2f} seconds")

```

Downloading file 0...
 Downloading file 1...
 Downloading file 2...
 Downloading file 3...
 Downloading file 4...
 Downloading file 5...
 Downloading file 6...
 Downloading file 7...
 Downloading file 8...
 Downloading file 9...
 Downloaded file 2: 1059 bytes
 Downloaded file 8: 1059 bytes
 Downloaded file 9: 1059 bytes
 Downloaded file 7: 1059 bytes
 Downloaded file 4: 1059 bytes
 Downloaded file 6: 1059 bytes
 Downloaded file 1: 1059 bytes
 Downloaded file 5: 1059 bytes
 Downloaded file 0: 1059 bytes
 Downloaded file 3: 1059 bytes
 Asynchronous downloading took 1.30 seconds

Figure 3: Asynchronous download output and timing

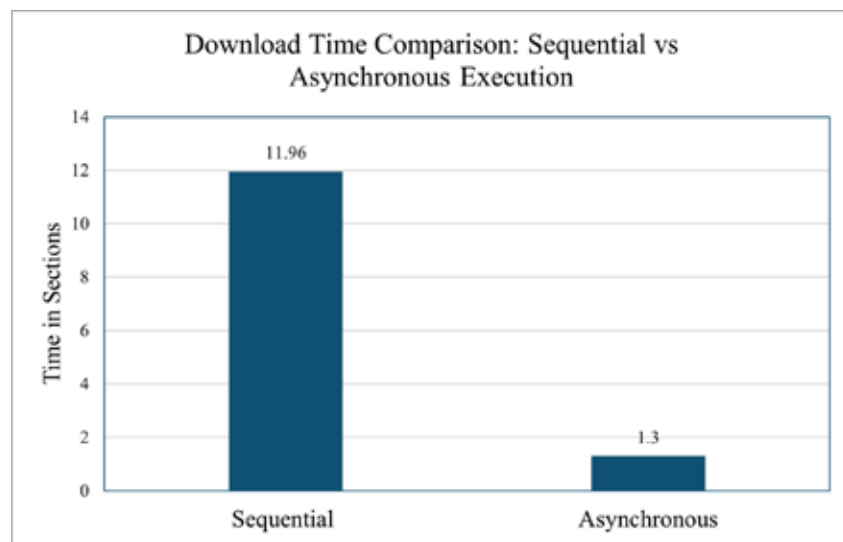


Figure 4: Representation of performance gain

one after the other. In contrast, the asynchronous method completed all downloads in just 1.30 seconds by launching all tasks concurrently using Python's `asyncio` and `aiohttp` libraries.

This performance gain is visually represented in Figure 4, where the bar graph compares the total execution times for both methods. The dramatic reduction in time highlights the power of asynchronous programming in efficiently handling I/O-bound operations.

Asynchronous programming is not just a clever optimisation — it is a foundational approach to building modern, scalable, and responsive Python applications. Whether you're developing a web server, creating a data pipeline, or writing a high-performance script that interacts with the network or filesystem, async techniques can help you do more in less time.

This article demonstrated that the benefits of async are not theoretical. The experiment of downloading a 10MB file ten times revealed how asynchronous programming can drastically reduce execution time, showing a nearly 10x improvement over sequential execution.

So the key takeaways are:

- Asynchronous code overlaps waiting time, reducing bottlenecks.
- It enables high-concurrency programs with minimal overhead.
- Achieves parallel-like performance without threads or multiprocessing.

In an I/O-bound world, mastering async is not optional — it's essential. **END** 🐧

By: S. Ratan Kumar

The author is a passionate user of open source and currently works as associate professor in the CSE Department of Anil Neerukonda Institute of Technology & Sciences, Visakhapatnam.